

Trabalho Prático 2: Estrutura de dados

Gabriel de Araújo Costa Gomes
2022102872

Departamento de Ciência da Computação; Universidade Federal de Minas Gerais
Belo Horizonte, Minas Gerais, Brasil; 28 de janeiro de 2025
gacg2004@ufmg.br

1 Introdução

O presente relatório descreve o desenvolvimento de um sistema de escalonamento hospitalar baseado em eventos discretos. O objetivo é simular a gestão de pacientes em um hospital, considerando múltiplas etapas, como triagem, atendimento médico, e procedimentos laboratoriais. O problema em questão requer o gerenciamento eficiente de eventos e recursos, garantindo que os pacientes sejam atendidos dentro de um fluxo controlado e priorizado.

A solução proposta utiliza estruturas de dados baseadas em *heaps* e algoritmos eficientes para gerenciar as prioridades dos eventos, minimizando os tempos de espera dos pacientes e otimizando o uso dos recursos hospitalares.

2 Método

A implementação baseia-se em uma estrutura de *MinHeap*, que organiza os eventos de acordo com o tempo de início e, em caso de empate, utiliza o ID do paciente como critério de desempate. Abaixo são descritas as principais estruturas e funções desenvolvidas:

2.1 Organização dos Arquivos

A organização dos arquivos do projeto segue a seguinte estrutura:

```
\TP
\src
    main.c
    escalonador.c
    paciente.c
    fila.c
    procedimento.c
\bin      # Arquivo executável
\obj      # Arquivos .o (objetos)
\include
    escalonador.h
    paciente.h
    fila.h
    procedimento.h
Makefile
```

- **Arquivo main.c:** Contém a função principal, onde o fluxo de execução é controlado. Ele realiza a leitura do arquivo de entrada, inicializa as estruturas de dados, insere e processa os eventos, além de exibir os resultados.
- **Arquivo escalonador.c:** Implementa as funções relacionadas ao gerenciamento do *MinHeap*, como a inserção, remoção, ajuste de elementos e ordenação pelo ID dos pacientes.
- **Arquivo paciente.c:** Contém as funções associadas à manipulação dos pacientes, incluindo o cálculo do tempo de chegada e a atualização dos estados.
- **Arquivo fila.c:** Implementa as funções para a manipulação de filas, como enfileirar, desenfileirar e verificar estados das filas de pacientes.
- **Arquivo procedimento.c:** Gerencia os procedimentos hospitalares, calculando os tempos médios de término e alocando recursos aos pacientes.

- **Arquivos de Cabeçalho (.h):** Contêm as declarações das funções e estruturas usadas em seus respectivos arquivos de implementação, garantindo modularidade e reutilização do código.
- **Makefile:** Define as regras de compilação do projeto, gerando os arquivos objeto e o executável final, além de incluir comandos para limpeza e reconstrução do projeto.

2.2 Ambiente de Execução

Os testes e experimentos foram realizados em uma máquina com as seguintes características:

- **Sistema Operacional:** Ubuntu 20.04 LTS.
- **Processador:** Intel Core™ i3-9100.
- **Memória RAM:** 4 GB.
- **Compilador:** GCC 10.3.0 com as opções `-Wall -g -fsanitize=address`.

2.3 Formato de Entrada e Saída

Entrada: O programa recebe como entrada um arquivo CSV contendo os dados a serem processados. O formato dos registros é o seguinte:

- As primeiras 6 linhas do arquivo contêm o tempo em horas e a quantidade de unidades disponíveis, respectivamente para: triagem, atendimento, medidas hospitalares, testes de laboratório, exames de imagem e instrumentos e medicamentos.
- As linhas subsequentes contêm os pacientes que vão entrar no hospital com o id, data de entrada, grau de prioridade e a quantidade de cada tipo de procedimento que irão realizar.

Saída: O programa gera como saída os registros processados e escalonados por cada estado e procedimento. Além disso, durante a análise experimental, o tempo de execução de cada etapa (carregamento, escalonamento e impressão) é exibido no terminal. Os resultados consolidados podem ser usados para avaliações de desempenho do sistema e melhorias no fluxo hospitalar.

2.4 Estruturas de Dados

- **MinHeap:** Estrutura responsável por armazenar e priorizar os eventos. Utiliza as funções de *heapify* para manter a propriedade do *heap*.
- **Evento:** Representa um evento na simulação, contendo informações como o tempo de início, tempo de fim, ID do paciente, e o estado atual do paciente.
- **Paciente:** Contém os atributos associados a cada paciente, como ID, estado atual, tempo de espera acumulado, e dados específicos do atendimento (exames, medidas hospitalares, etc.).

2.5 Funções Implementadas

- **inicializaHeap:** Inicializa a estrutura de *heap* com uma capacidade inicial.
- **ajustaParaCima/ajustaParaBaixo:** Garantem a manutenção da propriedade do *heap* após inserções ou remoções.
- **insereEvento:** Insere novos eventos no *heap*, garantindo a priorização correta.
- **ordenaID:** Ordena os eventos por ID utilizando o algoritmo *quicksort*.
- **retiraProximoEvento:** Remove o evento com maior prioridade (menor tempo de início) do *heap*.

3 Análise de Complexidade

Nesta seção, analisamos a complexidade de tempo e espaço das principais funções implementadas no projeto. As análises consideram os cenários de pior caso, melhor caso e caso médio, conforme aplicável.

3.1 Funções principais

3.1.1 inicializaHeap

- **Descrição:** Inicializa a estrutura de *heap* com uma capacidade inicial definida.
- **Complexidade de tempo:** $O(1)$, pois apenas aloca memória e inicializa os campos.
- **Complexidade de espaço:** $O(n)$, onde n é a capacidade inicial do *heap*.

3.1.2 ajustaParaCima e ajustaParaBaixo

- **Descrição:** Garantem a propriedade do *heap* ao ajustar elementos para cima ou para baixo.
- **Complexidade de tempo:** $O(\log n)$ no pior caso, onde n é o número de elementos no *heap*.
- **Complexidade de espaço:** $O(1)$, pois os ajustes ocorrem in-place.

3.1.3 expandirHeap

- **Descrição:** Expande a capacidade do *heap* ao atingir o limite, duplicando o espaço disponível.
- **Complexidade de tempo:** $O(n)$, onde n é o número de elementos atuais no *heap*, devido à cópia dos elementos para o novo espaço alocado.
- **Complexidade de espaço:** $O(n)$, para a nova alocação de memória.

3.1.4 insereEvento

- **Descrição:** Insere um novo evento no *heap*, mantendo a propriedade de prioridade.
- **Complexidade de tempo:** $O(\log n)$, devido ao ajuste para cima no *heap*.
- **Complexidade de espaço:** $O(1)$, já que a inserção ocorre in-place.

3.1.5 ordenaID (Quicksort)

- **Descrição:** Ordena os eventos por ID de paciente após a inserção.
- **Complexidade de tempo:**
 - Caso médio: $O(n \log n)$.
 - Melhor caso: $O(n \log n)$.
 - Pior caso: $O(n^2)$, devido ao particionamento degenerado.
- **Complexidade de espaço:** $O(\log n)$ no caso médio devido à recursão no *quicksort*.

3.1.6 retiraProximoEvento

- **Descrição:** Remove o menor elemento (raiz) do *heap* e ajusta os elementos para manter a propriedade de prioridade.
- **Complexidade de tempo:** $O(\log n)$, devido ao ajuste para baixo no *heap*.
- **Complexidade de espaço:** $O(1)$, pois a operação é realizada in-place.

3.1.7 exibeHeap

- **Descrição:** Exibe os eventos do *heap*, convertendo tempos de execução para um formato legível.
- **Complexidade de tempo:** $O(n)$, pois cada elemento é acessado uma vez.
- **Complexidade de espaço:** $O(1)$, pois nenhuma memória adicional significativa é alocada.

3.2 Complexidade geral

O tempo total de execução do programa é dominado pelas seguintes operações:

- Inserção de eventos no *heap*: $O(n \log n)$, onde n é o número total de eventos inseridos.
- Ordenação por ID (`ordenaID`): $O(n^2)$ no pior caso, quando o *quicksort* encontra particionamentos ruins.
- Processamento de eventos: $O(n \log n)$, considerando que cada evento é removido e ajustado no *heap*.
- Exibição dos resultados: $O(n)$, para percorrer todos os elementos.

A complexidade de espaço do programa é dominada pela estrutura do *heap* e pelos eventos armazenados, totalizando $O(n)$, onde n é o número de eventos processados.

4 Estratégias de Robustez

A implementação inclui mecanismos para garantir a robustez e tolerância a falhas:

4.1 Validação de entrada:

Verifica se os arquivos de entrada foram abertos corretamente e se os dados estão no formato esperado.

Gerenciamento de memória: Utiliza funções como `malloc`, `realloc`, e `free` para alocação dinâmica, com verificações de erro para evitar falhas de segmentação.

4.2 Resiliência em erros de inserção:

Expand automaticamente o *heap* quando atinge a capacidade, garantindo que novos eventos possam ser adicionados.

4.3 Mensagens de depuração

Para facilitar a identificação de problemas, o programa inclui mensagens de depuração em pontos críticos:

- Cada etapa do processamento exibe mensagens detalhadas sobre o progresso, como carregamento de arquivos, criação de índices, ordenação e destruição da estrutura.
- Mensagens de erro são específicas e indicam exatamente onde ocorreu a falha, facilitando o diagnóstico e a correção.

4.4 Manutenibilidade e extensibilidade

A modularidade do código, com funções bem definidas e separação de responsabilidades, contribui para a robustez do sistema. Alterações ou extensões podem ser realizadas com impacto mínimo, uma vez que cada função opera de maneira independente dentro de sua responsabilidade.

Com essas estratégias, o programa é capaz de operar de maneira segura, eficiente e resistente a falhas, mesmo em cenários adversos ou com entradas inesperadas.

5 Análise Experimental

Os experimentos foram realizados para avaliar o desempenho computacional do sistema em diferentes cenários. O tempo de execução, escalonamento, impressão e carregamento de dados foram medidos para várias quantidades de pacientes:

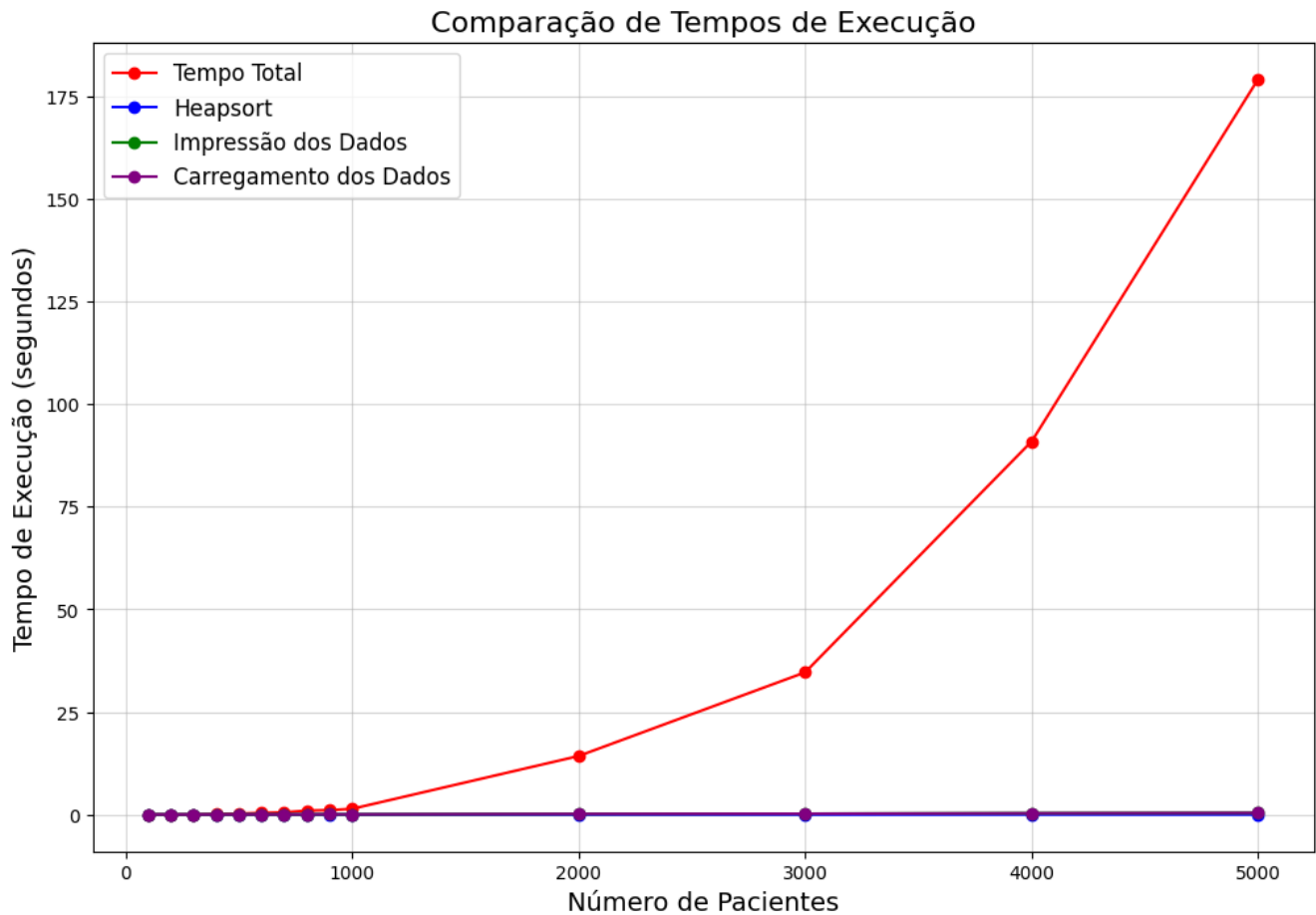


Figura 1: Tempos de execução

5.1 Explicação dos Resultados

5.1.1 Tempo Total

- Observação: O tempo total aumenta de forma não linear, especialmente quando o número de pacientes ultrapassa 1000.
- Causa provável: A principal causa é a combinação de múltiplas operações custosas, como o uso do quicksort no final do processo. Como o quicksort tem complexidade $O(n^2)$ no pior caso, ele pode aumentar significativamente o tempo de execução para grandes entradas.

Operações cumulativas de gerenciamento de eventos no heap também contribuem, mas a maior sobrecarga está no passo de ordenação.

5.1.2 Heapsort

- Observação: Os tempos de execução para o heapsort permanecem insignificantes para pequenas entradas e apenas começam a registrar valores mensuráveis após 4000 pacientes.
- Causa provável: O heapsort tem complexidade $O(n \log n)$ e é eficiente para ordenar grandes quantidades de dados. Portanto, ele não é o principal responsável pelo crescimento exponencial no tempo total.

5.1.3 Carregamento e Impressão dos Dados

- Observação: O carregamento e a impressão dos dados apresenta flutuações, mas mantém um crescimento relativamente mais estável do que o tempo total.

- Causa provável: A impressão envolve operações como conversão de dados (ex.: para datas legíveis) e manipulação de strings. Essas operações são $O(n)$, mas o custo acumulado pode crescer dependendo do número de elementos e da complexidade da formatação.

5.1.4 Conclusão

O principal fator para o aumento nos tempos de execução está no uso do quicksort durante o processamento do heap. Substituir o quicksort por algoritmos com melhor desempenho no pior caso, como o merge sort ou uma abordagem incremental, poderia melhorar significativamente o desempenho.

Os resultados mostram que o tempo de execução cresce significativamente para grandes entradas, sugerindo que a operação de *quicksort* introduz uma sobrecarga considerável no desempenho.

6 Conclusões

Este trabalho implementou um sistema de escalonamento hospitalar baseado em eventos discretos, utilizando *heaps* e algoritmos eficientes para gerenciar a prioridade dos eventos. Durante o desenvolvimento, foi possível observar a importância de estruturas de dados adequadas para lidar com grandes volumes de dados.

A análise experimental evidenciou que o uso do *quicksort* impacta negativamente o desempenho em cenários com muitos pacientes. Como aprendizado, reforça-se a necessidade de escolher algoritmos com bom desempenho no pior caso para aplicações críticas.

7 Bibliografia

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language*. Prentice Hall.
- Documentation: Biblioteca padrão *C Library Reference*.